

Лабораторная работа № 14

Программирование в командном процессоре ОС UNIX

Взваров Михаил, НПИбд-01-25

Лабораторная работа № 14

Программирование в командном процессоре ОС UNIX

Взваров Михаил

НПИбд-01-25

Цель работы

Получить практические навыки написания командных файлов в Bash:

- ▶ работа с системной справочной информацией;
- ▶ генерация случайных последовательностей;
- ▶ использование циклов и функций;
- ▶ синхронизация процессов с помощью файлового семафора.

Выполненные задания

В ходе лабораторной работы были реализованы три командных файла:

1. `myman.sh` — вывод справочной информации о команде;
2. `random_letters.sh` — генерация случайных букв;
3. `semaphore.sh` — синхронизация процессов с помощью lock-файла.

Скрипт myman.sh

Скрипт принимает имя команды и выполняет поиск соответствующей справочной страницы в каталоге `/usr/share/man/man1`.

Поддерживается поиск обычных и сжатых файлов справки.

Исходный код myman.sh

```
5 if [ $# -ne 1 ]; then
6     echo "Использование: $0 <команда>"
7     echo "Пример: $0 ls"
8     exit 1
9 fi
10
11 CMD="$1"
12
13 if [ ! -d "$MAN_DIR" ]; then
14     echo "Ошибка: каталог $MAN_DIR не найден"
15     exit 1
16 fi
17
18 MAN_FILE=""
19 for ext in "" ".gz" ".bz2" ".xz"; do
20     if [ -f "$MAN_DIR/${CMD}.${ext}" ]; then
21         MAN_FILE="$MAN_DIR/${CMD}.${ext}"
22         break
23     fi
24 done
25
26 if [ -z "$MAN_FILE" ]; then
27     echo "Справка для команды '$CMD' не найдена"
28     echo "Проверьте: man $CMD"
29     exit 1
30 fi
31
32 echo "=== Справка для команды '$CMD' ==="
33 echo "Источник: $MAN_FILE"
34 echo "-----"
35
36 if command -v less &> /dev/null; then
37     less "$MAN_FILE"
38 else
39     case "$MAN_FILE" in
40         *.gz) gunzip -c "$MAN_FILE" ;;
41         *.bz2) bunzip2 -c "$MAN_FILE" ;;
42         *.xz) unxz -c "$MAN_FILE" ;;
43         *) cat "$MAN_FILE" ;;
44     esac
45 fi
```

michailvzvarov@fedora:~/reports/lab14\$

Рис. 1: Листинг myman.sh

Результат работы myman.sh

Скрипт был запущен для команды `ls`.

Была найдена и открыта соответствующая справочная страница.

```
==> (lesspipe 2.22) append : to filename to view the original troff file
LS(1) User Commands

NAME
  ls - list directory contents

SYNOPSIS
  ls [OPTION]... [FILE]...

DESCRIPTION
  List information about the FILES (the current directory by default). Sort entries alphabetically if none of -cftuvSUX nor --sort is specified.
  Mandatory arguments to long options are mandatory for short options too.

  -a, --all
      do not ignore entries starting with .

  -A, --almost-all
      do not list implied . and ..

  --author
      with -l, print the author of each file

  -b, --escape
      print C-style escapes for nongraphic characters

  --block-size=SIZE
      with -l, scale sizes by SIZE when printing them; e.g., '--block-size=M'; see SIZE format below

  -B, --ignore-backups
      do not list implied entries ending with ~

  -c
      with -lt: sort by, and show, ctime (time of last change of file status information); with -l: show ctime and sort by name; otherwise: sort by ctime, newest

  -C
      list entries by columns

  --color[=WHEN]
      color the output WHEN; more info below

  -d, --directory
      list directories themselves, not their contents

/usr/share/man/man1/ls.1.gz
```

Рис. 2: Результат myman.sh

Скрипт random_letters.sh

Скрипт генерирует случайные последовательности латинских букв.

Для генерации используется встроенная переменная Bash `RANDOM`.

Параметрами задаются длина последовательности и режим использования заглавных букв.

Исходный код random_letters.sh

```
17 USE_UPPER=${2:-0}
18
19 echo "Генерация случайной последовательности длиной $LENGTH..."
20
21 if [ "$USE_UPPER" -eq 1 ]; then
22     for i in $(seq 1 $LENGTH); do
23         random_letter_upper
24     done
25 else
26     for i in $(seq 1 $LENGTH); do
27         random_letter
28     done
29 fi
30
31 echo ""
32
33 echo ""
34 echo "=== Разные варианты ==="
35
36 echo -n "Строчные: "
37 for i in $(seq 1 15); do
38     random_letter
39 done
40 echo ""
41
42 echo -n "Заглавные: "
43 for i in $(seq 1 15); do
44     random_letter_upper
45 done
46 echo ""
47
48 echo -n "Смешанные: "
49 for i in $(seq 1 20); do
50     if [ $((RANDOM % 2)) -eq 0 ]; then
51         random_letter
52     else
53         random_letter_upper
54     fi
55 done
56 echo ""
57
```

michailvzarov@fedora:~/reports/lab14\$

Рис. 3: Листинг random_letters.sh

Результат random_letters.sh

Были получены:

- ▶ случайная последовательность заданной длины;
- ▶ последовательность строчных букв;
- ▶ последовательность заглавных букв;
- ▶ смешанная последовательность.

```
Генерация случайной последовательности длиной 25...  
KFXES6TLPRQFWQEBEXOLGWJSK
```

```
=== Разные варианты ===
```

```
Строчные: hdpjxepqxfrngwx
```

```
Заглавные: TNDCFUNEQPPDSJ
```

```
Смешанные: eNYJLFKVJsQSjOp0ITva
```

```
michailvzvarov@fedora:~/reports/lab14$
```

Скрипт semaphore.sh

Скрипт демонстрирует синхронизацию нескольких процессов при доступе к общему ресурсу.

В качестве семафора используется файл:

`/tmp/semaphore.lock`

Принцип работы семафора

Если lock-файл удалось создать, процесс получает доступ к ресурсу.

Если lock-файл уже существует, другой процесс ожидает заданное время и повторяет попытку.

После завершения работы ресурс освобождается удалением lock-файла.

Исходный код semaphore.sh

```
1 #!/bin/bash
2
3 WAIT_TIME=${1:-5}
4 USE_TIME=${2:-3}
5 PROCESS_ID=${3:-1}
6 LOCK_FILE="/tmp/semaphore.lock"
7
8 log() {
9     echo "[$(date +%H:%M:%S)] [Процесс $PROCESS_ID] $1"
10 }
11
12 log "Запущен. Время ожидания: $WAIT_TIME сек, время использования: $USE_TIME сек"
13
14 while true; do
15     log "Пытаюсь захватить ресурс..."
16
17     if ( set -o noclobber; echo "$$" > "$LOCK_FILE" ) 2>/dev/null; then
18         # Ресурс захвачен
19         log "Ресурс захвачен! Использую в течение $USE_TIME секунд..."
20         sleep $USE_TIME
21
22         rm -f "$LOCK_FILE"
23         log "Ресурс освобождён"
24     else
25         log "Ресурс занят, ожидаю $WAIT_TIME секунд..."
26         sleep $WAIT_TIME
27     fi
28 done
29
```

mikhailvzvarov@fedora:~/reports/lab14\$

Рис. 5: Листинг semaphore.sh

Проверка синхронизации

Для проверки одновременно были запущены два экземпляра программы.

Один процесс захватил ресурс, а второй обнаружил занятость ресурса и ожидал его освобождения.

```
1 #!/bin/bash
2
3 WAIT_TIME=${1:-5}
4 USE_TIME=${2:-3}
5 PROCESS_ID=${3:-1}
6 LOCK_FILE="/tmp/semaphore.lock"
7
8 log() {
9     echo "[$(date +%H:%M:%S)] [Процесс $PROCESS_ID] $1"
10 }
11
12 log "Запущен. Время ожидания: $WAIT_TIME сек, время использования: $USE_TIME сек"
13
14 while true; do
15     log "Пытаюсь захватить ресурс..."
16
17     if ( set -o noclobber; echo "$$" > "$LOCK_FILE" ) 2>/dev/null; then
18         # Ресурс захвачен
19         log "Ресурс захвачен! Использую в течение $USE_TIME секунд..."
20         sleep $USE_TIME
21
22         rm -f "$LOCK_FILE"
23         log "Ресурс освобождён"
24     else
25         log "Ресурс занят, ожидаю $WAIT_TIME секунд..."
26         sleep $WAIT_TIME
27     fi
28 done
29
michailvzvarov@fedora:~/reports/lab14$ rm -f /tmp/semaphore.lock
michailvzvarov@fedora:~/reports/lab14$ cd ~/reports/lab14
chmod +x semaphore.sh
./semaphore.sh 2 5 1
[23:00:17] [Процесс 1] Запущен. Время ожидания: 2 сек, время использования: 5 сек
[23:00:17] [Процесс 1] Пытаюсь захватить ресурс...
[23:00:17] [Процесс 1] Ресурс захвачен! Использую в течение 5 секунд...
[23:00:22] [Процесс 1] Ресурс освобождён
[23:00:22] [Процесс 1] Пытаюсь захватить ресурс...
[23:00:22] [Процесс 1] Ресурс захвачен! Использую в течение 5 секунд...
```

Использованные средства Bash

В работе применялись:

- ▶ аргументы командной строки;
- ▶ условные операторы;
- ▶ циклы `for` и `while`;
- ▶ функции;
- ▶ переменная `RANDOM`;
- ▶ команда `sleep`;
- ▶ режим `noclobber`;
- ▶ работа с файлами и каталогами.

Результаты работы

В результате были созданы и проверены три Bash-скрипта.

Получены практические навыки:

- ▶ обработки аргументов;
- ▶ работы со справочными файлами UNIX;
- ▶ генерации случайных данных;
- ▶ организации взаимодействия процессов.

Вывод

В ходе лабораторной работы были закреплены навыки программирования в оболочке Bash.

Были реализованы собственный механизм просмотра справочной информации, генератор случайных последовательностей и файловый семафор для синхронизации процессов.

Цель лабораторной работы достигнута.